



Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 9

Software Testing



Ursinus
COLLEGE

Team Standup Meetings



Last Week

Implementation & Professional Coding Practices



Why Testing Is Hard

- Testing cannot prove the absence of bugs — only their presence
- Hidden assumptions: code does what you meant, not what you wrote
Example: off-by-one errors, null inputs, time zone edge cases
- The input space is effectively infinite — you can't test everything
- Edge cases are often unknown until a user finds them
Negative numbers, empty strings, concurrent users, network failures...
- Tests themselves can have bugs
- Knowing what to test requires understanding the requirements — which are often incomplete



r/programminghumor 14m



It be like that



Brenan Keller

@brenankeller



A QA engineer walks into a bar.
Orders a beer. Orders 0 beers.
Orders 99999999999 beers.
Orders a lizard. Orders -1 beers.
Orders a ueicbksjdhd.

First real customer walks in
and asks where the bathroom
is. The bar bursts into flames,
killing everyone.

1:21 PM · 30 Nov 18



54



1





Quick Reminder: The Cost of Bugs Over Time

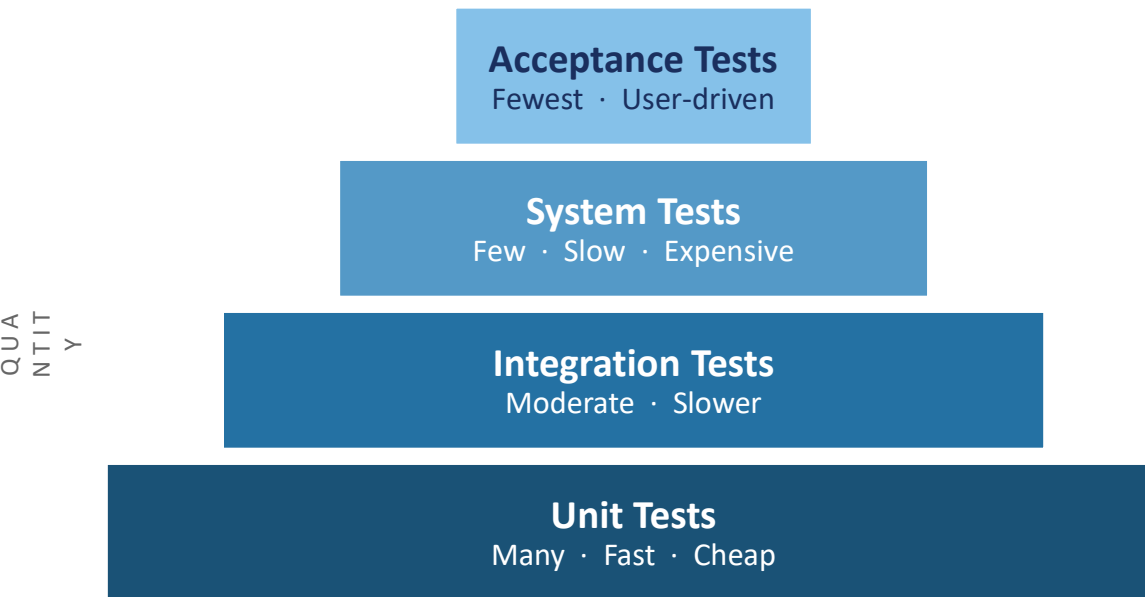
- Cost of fixing a bug grows roughly exponentially the later it's found
Requirements → Design → Code → Test → Production
- A bug found in production can cost 10–100× more than one caught in code review
- This is the core economic argument for investing in testing early
- We've covered this before — today we focus on how to actually do it



Levels of Testing

- Unit Testing — test a single function or class in isolation
Fast, deterministic, numerous. The foundation.
- Integration Testing — test how components interact
Does the service layer talk to the DB correctly? Does auth work end-to-end?
- System Testing — test the full application against requirements
Simulates real user scenarios across the whole stack
- Acceptance Testing — does the system satisfy the business/user need?
Often involves stakeholders, not just developers
- The Testing Pyramid: many unit tests, fewer integration tests, fewer still system tests

The Testing Pyramid



Why a pyramid?

Shape reflects quantity. Thousands of unit tests, hundreds of integration tests, dozens of system tests, a handful of acceptance tests.

The anti-pattern: Ice Cream Cone

Many teams invert this — heavy on manual system tests, almost no unit tests. Slow, brittle, expensive. A very common real-world failure mode.

Speed matters

A test suite developers cannot run quickly is one they stop running. Unit tests in milliseconds; system tests in minutes. Invest accordingly.



Unit Testing Principles

- A unit test exercises one function or class in complete isolation
- Isolation: no database calls, no network, no file system — use mocks/stubs for dependencies
- Determinism: same input must always produce the same output
Flaky tests (pass sometimes, fail sometimes) are worse than no tests — they destroy trust
- Speed: unit tests should run in milliseconds — the whole suite in seconds
- Readability: a test is documentation — the next developer should understand intent at a glance
- F.I.R.S.T. principle: Fast · Isolated · Repeatable · Self-validating · Timely
- One logical assertion per test — test one concept, not everything at once

Anatomy of a Unit Test

```
// The function we want to test
function add(a, b) {
  return a + b;
}

// The unit test (Jest / JavaScript)
test('adds two positive numbers', () => {
  // Arrange
  const a = 3, b = 4;
  // Act
  const result = add(a, b);
  // Assert
  expect(result).toBe(7);
});
```

Test name

Describes the behavior being tested. Read it like a sentence: 'it adds two positive numbers.'

Arrange

Set up the inputs and any state needed for the test.

Act

Call the function under test. Usually just one line.

Assert

Verify the result is what you expected. If this fails, the test fails.

expect().toBe()

Jest's assertion syntax. toBe checks strict equality. Other matchers: toEqual, toBeTruthy, toThrow...

Assertions: Checking What You Got

An assertion is a claim about the program's state. A failing assertion = a failing test.

```
// Equality
expect(add(2, 3)).toBe(5);
expect(getUser()).toEqual({ id: 1, name: 'Alice' });

// Truthiness
expect(isValid('hello')).toBeTruthy();
expect(isValid('')).toBeFalsy();

// Errors & exceptions
expect(() => divide(10, 0)).toThrow('Division by zero');

// Arrays & strings
expect([1, 2, 3]).toContain(2);
expect('hello world').toMatch(/world/);

// Null / undefined
expect(findUser(999)).toBeNull();

//Java equivalent (JUnit 5):
assertEquals(5, add(2,3));
assertThrows(ArithmeticException.class, () -> divide(10,0));
```

A More Realistic Unit Test Suite – JavaScript (Jest)

```
// shoppingCart.js
function applyDiscount(price, pct) {
  if (pct < 0 || pct > 100)
    throw new Error(
      'Invalid discount');
  return price * (1 - pct / 100);
}
```

```
// shoppingCart.test.js
describe('applyDiscount', () => {

  test('applies 10% off correctly', () => {
    expect(applyDiscount(100, 10))
      .toBe(90);
  });

  test('zero discount returns full price', () => {
    expect(applyDiscount(200, 0)).toBe(200);
  });

  test('100% discount gives zero', () => {
    expect(applyDiscount(50, 100)).toBe(0);
  });

  test('throws on invalid discount', () => {
    expect(() => applyDiscount(100, -5))
      .toThrow('Invalid discount');
  });
});
```

Same Tests in Java – JUnit 5

Different syntax, identical concepts: Arrange / Act / Assert, one behavior per test, descriptive names.

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class ShoppingCartTest {

    @Test
    @DisplayName("applies 10% off correctly")
    void appliesTenPercentDiscount() {
        assertEquals(90.0, applyDiscount(100.0, 10));
    }

    @Test
    @DisplayName("throws on invalid discount")
    void throwsOnInvalidDiscount() {
        assertThrows(IllegalArgumentException.class,
            () -> applyDiscount(100.0, -5));
    }
}
```



Exercise: Find the Missing Tests

Given this JavaScript function:

```
function calculateShipping(weight, destination) {  
  if (destination === 'international') return weight * 4.5;  
  if (weight > 50) return 15.0;  
  return weight * 0.75;  
}
```

The team wrote ONE test:

```
expect(calculateShipping(10, 'domestic')).toBe(7.50)
```

What test cases are missing? What could go wrong?

Think about: edge cases, invalid inputs, boundary values, hidden assumptions



Integration Testing

- Tests interactions between components that were unit-tested in isolation
- Catches interface mismatches: wrong data formats, unexpected nulls, timing issues
- Common integration test targets:
 - Service $\leftrightarrow$ Database (does the ORM query return what we expect?)
 - API $\leftrightarrow$ Auth layer (do tokens get validated correctly?)
 - Microservice A $\leftrightarrow$ Microservice B (do they agree on the contract?)
- Slower than unit tests — needs real (or realistic) infrastructure
- Use test databases, containers (Docker), or in-memory substitutes
- Contract testing (e.g., Pact) is a modern approach for microservice boundaries



System Testing

- Tests the fully integrated application as a whole, from the user's perspective
- Validates end-to-end workflows against requirements
 - Example: user registers → verifies email → logs in → places order → gets confirmation
- Often automated using browser/UI automation tools (Selenium, Playwright, Cypress)
- Also includes:
 - Performance / load testing (does it hold up under 10,000 users?)
 - Security testing (are endpoints protected? Is data validated?)
 - Usability and compatibility testing
- Slow and expensive to run — run less frequently (nightly, pre-release)
- High value for catching regressions across the full stack
- Manual UI testing — testers follow scripts to verify layout, responsiveness, and visual correctness against a design spec or mockup. Catches things automated tests miss: misaligned elements, wrong fonts, broken on mobile.
 - Limitation: slow, subjective, doesn't scale
- Automated visual regression testing — tools like Percy, Chromatic, or Playwright screenshot comparisons take a baseline screenshot and flag pixel-level differences on subsequent runs
 - Catches unintended visual changes — e.g. a CSS change that shifts a button 10px on every page



Acceptance Testing

- Validates that the system satisfies the actual requirements from the user's perspective
- The question being asked: "Does this software do what we promised it would do?"
- Written against functional requirements — one test per requirement is a good starting point
- Two forms:
 - User Acceptance Testing (UAT): a real (non-technical) user follows a script to exercise a feature and verifies the result
 - Automated Acceptance Testing: scripts that simulate user workflows end-to-end (e.g. Cucumber, FitNesse)
- A UAT script describes: preconditions, steps to follow, and expected observable outcome
Example: "Log in with valid credentials. Expected: dashboard loads and username appears in top-right corner."
- Acceptance tests are the last line of defense — they confirm the right thing was built, not just that it was built correctly



What Would You Test?

- Scenario: You are the QA lead for a login/authentication system.
- The system supports: username/password login, OAuth (Google/GitHub), MFA via SMS, password reset via email, account lockout after 5 failed attempts.
- What are the most important things to test, at each level (Unit, Integration, System)?



Black-Box vs White-Box Testing

- Black-Box Testing: test behavior without knowledge of internal implementation
 - Tester only knows inputs and expected outputs
 - Closer to the user's perspective; implementation-agnostic
 - Used in system testing, acceptance testing, and by external QA teams
- White-Box Testing: test with full knowledge of internal code and structure
 - Can design tests to cover specific branches, paths, conditions
 - Enables coverage metrics (line, branch, path coverage)
 - Used heavily in unit and integration testing
- Grey-Box: partial knowledge — common in real-world practice
- Neither is better — they're complementary perspectives



Static Analysis

- Analyzes source code without executing it
- Catches bugs, style violations, and security vulnerabilities at write-time or build-time
- Examples:
 - Linters: ESLint, Pylint, Checkstyle — enforce style and flag suspicious patterns
 - Type checkers: TypeScript, mypy (Python) — catch type errors before runtime
 - SAST (Static Application Security Testing): SonarQube, Semgrep, Checkmarx
- SAST tools specifically target security: SQL injection patterns, hardcoded credentials, insecure crypto usage
- Runs fast — integrates easily into CI pipelines and IDEs
- Limitation: high false positive rate; can't catch logic errors or runtime behavior
- Think of it as a very thorough compiler that also reads for security



Dynamic Analysis & Security Testing

- Dynamic analysis: examines program behavior during execution
- Catches issues that only appear at runtime: memory leaks, race conditions, injection attacks
- Types:
 - DAST (Dynamic Application Security Testing): tools like OWASP ZAP, Burp Suite — probe a running app for vulnerabilities
 - Fuzzing: automatically generate malformed/unexpected inputs to find crashes (AFL, libFuzzer)
 - Penetration Testing: human-driven adversarial testing of a live system
- Pen testing goes beyond tools — testers think like attackers
 - Common targets: auth flows, session management, input validation, access control
- SAST + DAST together = much stronger security posture than either alone
- In practice: SAST in the IDE/CI, DAST in staging, pen test before major releases



Writing Testable Code

- Testability is a design property — it doesn't happen by accident
- Single Responsibility Principle: a function that does one thing is easy to test
A 300-line function that reads a file, parses it, validates it, and writes to a DB is hard to test at any level
- Avoid global state and hidden dependencies
If your function reaches into a global config or a singleton, tests become unpredictable
- Prefer pure functions where possible: same input → same output, no side effects
- Separate I/O from logic: the business logic shouldn't know about the database
Test the logic; mock the I/O
- Design for dependency injection — pass dependencies in, don't create them inside



Dependency Injection & Mocking

- Dependency Injection (DI): pass dependencies into a component rather than creating them inside
 - Without DI: function creates its own DB connection — impossible to test in isolation
 - With DI: DB connection passed in — in tests, pass in a mock instead
- Mocks and Stubs:
 - Stub: returns a fixed value (simulates a dependency's behavior)
 - Mock: also verifies how the dependency was called (did we call save() exactly once?)
- Most languages have mocking frameworks: Jest (JS), Mockito (Java), unittest.mock (Python)
- Test doubles let you test components that depend on things that are slow, external, or nondeterministic
- Warning: over-mocking can make tests that pass even when the real system is broken



Test Case Design

- A test case specifies input, preconditions, expected output, and postconditions
- Equivalence Partitioning: group inputs into classes that should behave the same way
Don't write 50 tests for 50 different valid ages — write one for any valid age, one for invalid
- Boundary Value Analysis: bugs cluster at the edges of valid input ranges
If valid range is 1–100, test: 0, 1, 2, 99, 100, 101
- Decision Table Testing: useful for functions with multiple conditional inputs
- State Transition Testing: for systems with states (e.g., order workflow: pending → paid → shipped → delivered)
- Error guessing: use experience to predict likely failure points
Empty string, null, zero, max int, special characters, SQL/HTML injection



Exercise: Write the Test Cases

Function spec: `validate_password(password: str) → bool`

Rules: Must be 8–64 characters. Must contain at least one uppercase letter, one digit, and one special character (!@#\$%^&*).

Must NOT contain spaces.

1. Identify your equivalence partitions
2. Identify your boundary values
3. Come up with at least 8 concrete test cases in the format:
input → expected output (and why)

Bonus: what ambiguities do you notice in the spec?



Common Testing Mistakes

- Testing too late — writing tests after the feature is done (or not at all)
By then, the code isn't designed for testability
- Only testing the happy path
- Brittle tests — tests tightly coupled to implementation details
A test that breaks every time you rename a variable is noise, not signal
- Testing implementation, not behavior
Test what the function does, not how it does it
- Ignoring flaky tests instead of fixing them
A green CI that nobody trusts is worse than useless
- Chasing 100% code coverage as a goal in itself
Coverage measures what was executed, not what was verified
- Not running tests automatically (relying on developers to remember)

Code Coverage

What is coverage?

The % of your code that is executed when your test suite runs. Measured by the framework instrumenting the code at build time.

Line (Statement) Coverage

Was each line executed at least once? The most basic metric — easy to achieve, easiest to game.

Branch Coverage

Was each branch of every if/else/switch taken? Stronger — requires testing both the true and false path of every condition.

Path Coverage

Every possible execution path exercised? Theoretically complete — practically impossible (paths grow exponentially with conditions).

The Coverage Trap

```
function divide(a, b) {  
  if (b === 0) throw new Error(  
    'Division by zero');  
  return a / b;  
}  
  
// Passes. Has coverage. Checks nothing.  
test('divides', () => {  
  divide(10, 2); // no assertion!  
});
```

No assertion = no verification. The line ran but nothing was checked. Passes even if divide() returns undefined. 100% line coverage $\neq$ 100% branch coverage. divide(10,2) never tests the b===0 branch — that throw is never reached. Use coverage as a floor, not a target. 80% is a reasonable minimum bar. Optimizing for the number produces hollow tests.



TDD: Tests as Specification

- Test-Driven Development: write the test before the code
- The TDD cycle: Red → Green → Refactor
 - Red: write a failing test that describes the behavior you want
 - Green: write the minimum code to make it pass
 - Refactor: clean up the code, keeping tests green
- Benefits:
 - Forces you to think about the interface before the implementation
 - Every line of code has a test by construction
 - Tests become a living specification of intended behavior
- Critique: can feel slow; can be awkward for exploratory or UI code
- Even if you don't do strict TDD, writing tests early dramatically improves design



Testing in Agile & CI/CD Pipelines

- Agile teams test continuously, not at the end of a phase
- Continuous Integration (CI): every commit triggers an automated test run
Tools: GitHub Actions, Jenkins, CircleCI, GitLab CI
- A failing test blocks the merge — no broken code reaches main
- The shift-left principle: move testing earlier in the development process
Static analysis in the IDE, unit tests on commit, integration tests on PR, system tests nightly
- Whole-team quality: QA is not a separate phase — everyone owns testing
Developers write unit and integration tests; QA focuses on higher-level and exploratory testing
- Test environments: dev → staging → production mirrors the real system at each stage
- Canary releases and feature flags: deploy to a small % of users first, watch for failures



Key Takeaways

- Testing is a design discipline — testable code is better code
- Test at multiple levels: unit, integration, system — each catches different things
- Use both black-box and white-box approaches — they're complementary
- Static analysis catches bugs before runtime; DAST and pen testing find runtime and security issues
- Design test cases systematically: equivalence classes, boundary values, error guessing
- Common mistakes: testing late, happy-path only, brittle tests, ignoring flaky tests
- TDD inverts the workflow — tests drive design
- In agile teams: testing is continuous, automated, and everyone's responsibility
- A test suite is a safety net — it lets you refactor and ship with confidence



Next Week:

CI/CD in Practice: Pipelines, GitHub Actions & Deployment Strategies



Assignment: Weekly Standup Reflection

<https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection>

(Individual Assignment)

Due Feb 26 (and every week thereafter)



Assignment: The Overdose

<https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose>

(Individual Assignment)

Due Apr 23